AP COMPUTER SCIENCE A

Exam Information


elloWorld.ja
*/
public class He
{
pu
ain

Exam Overview

*The AP Computer Science A Exam assesses student understanding of the computational thinking practices and learning objectives outlined in the course framework. The exam is 3 hours long and includes 42 multiple-choice questions and 4 free-response questions. As part of the exam, students will be given the Java Quick Reference (see **Appendix**), which lists accessible methods from the Java library that may be included on the exam. The details of the exam, including exam weighting and timing, can be found below:*

Section	Question Type	Number of Questions	Exam Weighting	Timing
I	**Multiple-choice questions**	42	55%	90 minutes
II	**Free-response questions**	4	45%	90 minutes
	Question 1: Methods and Control Structures (7 points)			
	Question 2: Class Design (7 points)			
	Question 3: Data Analysis with `ArrayList` (5 points)			
	Question 4: 2D Array (6 points)			

How Student Learning Is Assessed on the AP Exam

Section I: Multiple-Choice

The multiple-choice section of the AP Computer Science A Exam includes discrete and set-based questions.

All four units of the course are assessed in the multiple-choice section of the AP Exam with the following exam weighting:

Units of Instruction	Approximate Exam Weighting for Multiple-Choice Section
Unit 1: Using Objects and Methods	**15–25%**
Unit 2: Selection and Iteration	**25–35%**
Unit 3: Class Creation	**10–18%**
Unit 4: Data Collections	**30–40%**

All five computational thinking practices are assessed in the multiple-choice section of the AP Exam with the following approximate exam weighting:

Computational Thinking Practices	Approximate Exam Weighting for Multiple-Choice Section
Practice 1: Design Code	**2–10%**
Practice 2: Develop Code	**22–38%**
Practice 3: Analyze Code	**37–53%**
Practice 4: Document Code and Computing Systems	**10–15%**
Practice 5: Use Computers Responsibly	**2–10%**

Section II: Free-Response

The second section of the AP Computer Science A Exam includes four free-response questions, all of which assess Computational Thinking Practice 2: Develop Code. All three skills within this practice (2.A, 2.B, 2.C) are assessed across the four free-response questions, with the following skill focus for each question:

Free-Response Question 1: Methods and Control Structures. Students will write two methods or one constructor and one method of a given class based on provided specifications and examples. In Part A (4 points), the method or constructor will require students to write iterative or conditional statements, or both, as well as statements that call methods in the specified class. In Part B (3 points), the method or constructor will require calling `String` methods.

Free-Response Question 2: Class Design. Students will be instructed to design and implement a class based on provided specifications and examples. A second class might also be included. Students will be provided with a scenario and specifications in the form of a table demonstrating ways to interact with the class and the results. The class must include a class header, instance variables, a constructor, a method, and implementation of the constructor and required method.

Free-Response Question 3: Data Analysis with `ArrayList`. Students will be provided with a scenario and its associated class(es). Students will write one method of a given class based on provided specifications and examples. The method requires students to use, analyze, and manipulate data in an `ArrayList` structure.

Free-Response Question 4: 2D Array. Students will be provided with a scenario and its associated class(es). Students will write one method of a given class based on provided specifications and examples. The method requires students to use, analyze, and manipulate data in a 2D array structure.

Task Verbs Used in Free-Response Questions

The following task verbs are commonly used in the free-response questions:

Assume: Suppose to be the case without any proof or need to further address the condition.

Complete (program code): Express in print form the proper syntax to represent a described algorithm or program given part of the code.

Implement/Write: Express in print form the proper syntax to represent a described algorithm or program.

Sample Exam Questions

The sample exam questions that follow illustrate the relationship between the course framework and the AP Computer Science A Exam and serve as examples of the types of questions that appear on the exam. These sample questions do not represent the full range and distribution of items on an official AP Computer Science A Exam. After the sample questions is a table that shows which skill, learning objective(s), and essential knowledge statement(s) each question assesses. The table also provides the answers to the multiple-choice questions.

Section I: Multiple-Choice

1. Consider the following code segment.

```
double q = 15.0;
int r = 2;

double x = (int) (q / r);
double y = q / r;

System.out.println(x + " " + y);
```

What is printed as a result of executing this code segment?

(A) `7.0 7.0`

(B) `7.0 7.5`

(C) `7.5 7.0`

(D) `7.5 7.5`

2. Consider the following code segment.

```
int x = ((int) (Math.random() * 10)) + 5;
int y = ((int) (Math.random() * 5)) + 10;
System.out.println(x + " " + y);
```

Which of the following could be printed as a result of executing this code segment?

(A) `8 15`

(B) `9 12`

(C) `10 7`

(D) `15 11`

3. Consider the following `Date` class.

```
public class Date
{
   private String month;
   private int day;

   public Date(String m, int d)
   { /* implementation not shown */ }

   /* Class contains no other constructors.
      Other methods are not shown. */
}
```

Which of the following code segments, appearing in a class other than `Date`, will correctly create an instance of a `Date` object?

(A) `Date birthday = new Date("September", "5th");`

(B) `Date birthday = new Date("September", 5);`

(C) `Date birthday = new Date("September 5");`

(D)
```
Date birthday = new Date();
   birthday.month = "September";
   birthday.day = 5;
```

4. Consider the following code segment.

```
String str1 = "LMNOP";
String str2 = str1.substring(3);
str2 += str1.substring(2, 3);
```

What is the value of `str2` after executing this code segment?

(A) `"OPN"`

(B) `"OPNO"`

(C) `"NOPM"`

(D) `"NOPMN"`

5. The following method is intended to return `"Renaissance"` for years from 1400 to 1600, inclusive, `"Medieval"` for years from 400 to 1399, inclusive, or `"Other"` for any other year. In the method, the parameter represents a year.

```
public static String getCategory(int year)
{
   String category = "";
   /* missing code */
   return category;
}
```

Which of the following code segments can replace `/* missing code */` so that the `getCategory` method works as intended?

(A)
```
if (year > 1600 || year < 400)
{
   category = "Other";
}
else if (year >= 1400)
{
   category = "Renaissance";
}
else if (year >= 400)
{
   category = "Medieval";
}
```

(B)
```
if (year > 1600 || year < 400)
{
   category = "Other";
}
if (year >= 1400)
{
   category = "Renaissance";
}
if (year >= 400)
{
   category = "Medieval";
}
```

(C)
```
if (year >= 1400)
{
   category = "Renaissance";
}
else if (year >= 400)
{
   category = "Medieval";
}
else
{
   category = "Other";
}
```

(D)
```
if (year >= 1400)
{
   category = "Renaissance";
}
if (year >= 400)
{
   category = "Medieval";
}
else
{
   category = "Other";
}
```

6. Assume that `x`, `y`, and `z` are `boolean` variables that have been properly declared and initialized.

   Which of the following expressions is equivalent to `!x || !y || z`?

   (A) `!(x && y) && z`

   (B) `!(x && y) || z`

   (C) `!(x || y) && z`

   (D) `!(x || y) || z`

7. Consider the following code segment.

```
for (int j = 4; j > 0; j--)
{
   for (/* missing code */)
   {
      System.out.print(k + " ");
   }
   System.out.println();
}
```

   This code segment is intended to produce the following output.

```
0 1 2 3
0 1 2
0 1
0
```

   Which of the following can be used to replace `/* missing code */` so that this code segment works as intended?

   (A) `int k = 0; k < j; k++`

   (B) `int k = 0; k <= j; k++`

   (C) `int k = j; k > 0; k--`

   (D) `int k = j; k >= 0; k--`

8. Consider the following code segment.

```
int j = 1;
while (j < 4)
{
   for (int k = 0; k <= 4; k++)
   {
      System.out.println("hello");   // line 6
   }
   j++;
}
```

   How many times will the statement in line 6 be executed as a result of running this code segment?

   (A) 12

   (B) 15

   (C) 16

   (D) 20

9. A programmer is creating a `Movie` class that will contain attributes for a movie's title and its average user rating on a scale from `0.0` to `5.0`. The class must also contain methods to allow the attributes to be accessed outside the class.

   A class design diagram for the `Movie` class will contain three sections. The first section contains the class name, the second section contains two instance variables and their data types, and the third section contains two methods and their return types. The `+` symbol indicates a `public` designation, and the symbol `-` indicates a `private` designation.

   Which of the following diagrams represents the most appropriate design for the `Movie` class?

(A)
```
            Movie
+ title : String
+ rating : double
- getTitle() : String
- getRating() : double
```

(B)
```
            Movie
+ title : String
+ rating : int
- getTitle() : String
- getRating() : int
```

(C)
```
            Movie
- title : String
- rating : double
+ getTitle() : String
+ getRating() : double
```

(D)
```
            Movie
- title : String
- rating : int
+ getTitle() : String
+ getRating() : int
```

10. A programmer developed a method and wants to allow other programmers to use the method in other programs without raising intellectual property concerns. Which of the following actions is most likely to support this goal?

(A) Designating the method as `public` in the method header

(B) Designating the method as `static` in the method header

(C) Publishing the code for the method as open source

(D) Removing all personal information from the data used by the method

11. The following class declarations are used to represent information about an apartment and a duplex, which is a building containing two apartments.

```
public class Apartment
{
   public int calculateRent()
   { /* implementation not shown */ }

   private String getTenant()
   { /* implementation not shown */ }

   /* There may be instance variables, methods,
      and constructors that are not shown. */
}

public class Duplex
{
   private Apartment unitOne;
   private Apartment unitTwo;

   public void printInfo()
   {
      System.out.println( /* missing code */ );
   }

   /* There may be instance variables, methods,
      and constructors that are not shown. */
}
```

Which of the following replacements for `/* missing code */` will compile without error?

(A) `Apartment.calculateRent()`

(B) `Apartment.getTenant()`

(C) `unitOne.calculateRent()`

(D) `unitTwo.getTenant()`

12. Consider the following class declaration.

```
public class Point
{
   private int xCoor;  // the x-coordinate of the point
   private int yCoor;  // the y-coordinate of the point

   public Point(int x, int y)
   {
      xCoor = x;
      yCoor = y;
   }

   public int getXCoor()
   {
      return xCoor;
   }

   public int getYCoor()
   {
      return yCoor;
   }
   public boolean isEquiv(Point otherP)
   {
      return /* missing expression */;
   }
}
```

Two `Point` objects are considered equivalent if they have the same *x* and *y* coordinates. The `isEquiv` method is intended to return `true` if this `Point` object is equivalent to the `Point` object parameter and return `false` otherwise.

Which of the following expressions can replace `/* missing expression */` so that the method works as intended?

(A) `getXCoor() == xCoor && getYCoor() == yCoor`

(B) `otherP.xCoor == x && otherP.yCoor == y`

(C) `otherP.getXCoor() == getXCoor() &&`
`otherP.getYCoor() == getYCoor()`

(D) `otherP == this`

13. A researcher wants to investigate which age group (under 18 years old, or 18 years old and over) is more likely to have downloaded a certain application from an online store. Each customer of the store has a unique ID number. The researcher has the following data sets available.

- Data set 1 contains an entry for each application available from the store. Each entry includes the name of the application and the total number of customers who have downloaded it.
- Data set 2 contains an entry for each application available from the store. Each entry includes the ID number and age of the customer who most recently downloaded the application.
- Data set 3 contains an entry for each customer of the store. Each entry includes the customer's ID number and the customer's age.
- Data set 4 contains an entry for each customer of the store. Each entry includes the customer's ID number and the names of all the applications downloaded by the customer.

Which two data sets can be combined and analyzed to determine the desired information?

(A) Data sets 1 and 2

(B) Data sets 2 and 3

(C) Data sets 2 and 4

(D) Data sets 3 and 4

14. Consider the following method, which is intended to return the maximum integer value contained in `ranges`.

```
/** Precondition: ranges contains at least one element. */
public int findMaxValue(int[] ranges)
{
   int max = 0;                 // line 4
   for (int value : ranges)
   {
      if (value > max)          // line 7
      {
         max = value;           // line 9
      }
   }
   return max;
}
```

This method does not always work as intended. Which of the following changes can be made so that the code segment works as intended?

(A) Changing line 4 to `int max = ranges[0];`

(B) Changing line 4 to `int max = Integer.MAX_VALUE;`

(C) Changing line 7 to `if (value < max)`

(D) Changing line 9 to `value = max;`

15. A text file named `roster.txt` has the following contents. Assume that the file contains no spaces.

```
Cohen-Isabel
Lee-Cindy
Patel-Anisha
Sanchez-Michael
```

In the following code segment, a valid `Scanner` object named `scan` has been created to read from the text file.

```
File myText = new File("roster.txt");
Scanner scan = new Scanner(myText);

ArrayList<String> usernames = new ArrayList<String>();

/* missing code */
```

This code segment is intended to assign `["ICohen", "CLee", "APatel", "MSanchez"]` to `usernames`.

Which of the following can replace `/* missing code */` so that this code segment works as intended?

(A)
```
while (scan.hasNext())
{
   String temp = scan.next().substring(0, 1);
   usernames.add(temp + scan.next());
}
```

(B)
```
while (scan.hasNext())
{
   String one = scan.next();
   String two = scan.next();
   usernames.add(two.substring(0, 1) + one);
}
```

(C)
```
while (scan.hasNext())
{
   String[] temp = scan.next().split("-");
   usernames.add(temp[1].substring(0, 1) + temp[0]);
}
```

(D)
```
while (scan.hasNext())
{
   String[] temp = scan.next().split("-");
   usernames.add(temp[0].substring(0, 1) + temp[1]);
}
```

16. Consider the following method.

```
public void reviseList(ArrayList<Integer> list)
{
   for (int i = 0; i < list.size(); i++)
   {
      int temp = list.get(i);
      if (temp % 2 == 1)
      {
         list.set(i, temp + 1);
      }
   }
}
```

The `reviseList` method is called with an `ArrayList` containing the values `[1, 2, 3, 3, 2, 1]`. What will be the contents of the `ArrayList` after executing this method?

(A) `[1, 3, 3, 3, 3, 1]`

(B) `[1, 3, 3, 4, 2, 2]`

(C) `[2, 2, 4, 4, 2, 2]`

(D) `[2, 3, 4, 4, 3, 2]`

17. Consider the following code segment.

```
int[][] arr2D = /* missing initialization */;
int sum = 0;

for (int j = 0; j < arr2D.length; j++)
{
   sum += arr2D[j][2];
}

System.out.println(sum);
```

Which of the following replacements for `/* missing initialization */` will cause the code segment to print the value `3`?

(A)
```
{{0, 0, 1},
 {0, 0, 2},
 {0, 0, 1}}
```

(B)
```
{{0, 0, 0},
 {0, 1, 2},
 {0, 0, 0}}
```

(C)
```
{{0, 0, 0},
 {0, 0, 0},
 {1, 1, 1},
 {0, 0, 0}}
```

(D)
```
{{0, 0, 1, 0},
 {0, 0, 1, 0},
 {0, 0, 1, 0}}
```

18. Consider the following code segment.

```
int[][] arr = {{10, -7, 19, 14},
               {-3, -2, -6, 11},
               {-9, 12, 10, -1}};
int ans = 1;
for (int[] row : arr)
{
   for (int value : row)
   {
      if (value > 0)
      {
         ans = ans * value;
      }
   }
}
System.out.println(ans);
```

Which of the following best describes the behavior of the code segment?

(A) It prints the product of all the elements in `arr`.

(B) It prints the product of all the negative elements in `arr`.

(C) It prints the product of all the positive elements in `arr`.

(D) It prints nothing because of a run-time error.

19. The following method is a correct implementation of the selection sort algorithm. The method sorts the elements of `arr` so that they are in order from least to greatest.

```
public static void selectionSort(int[] arr)
{
   for (int j = 0; j < arr.length - 1; j++)
   {
      int minIndex = j;
      for (int k = j + 1; k < arr.length; k++)
      {
         if (arr[k] < arr[minIndex])
         {
            minIndex = k;
         }
      }

      int temp = arr[j];
      arr[j] = arr[minIndex];
      arr[minIndex] = temp;
      /* end of outer loop */
   }
}
```

Assume that `selectionSort` has been called with an `int[]` argument that has been initialized with the following contents.

`{40, 30, 50, 60, 10, 20}`

What will the contents of `arr` be after three iterations of the outer loop (i.e., when `j == 2` at the point indicated by `/* end of outer loop */`)?

(A) `{10, 20, 30, 40, 60, 50}`

(B) `{10, 20, 30, 60, 40, 50}`

(C) `{10, 20, 50, 60, 40, 30}`

(D) `{10, 30, 50, 60, 40, 20}`

20. Consider the following method.

```
public static void mystery(int j, int k)
{
   if (j > k)
   {
      mystery(j – 2, k + 2);
   }
   System.out.print(j + " ");
}
```

What is printed as a result of the call `mystery(10, 0)`?

(A) `10 8 6 4`

(B) `10 8 6`

(C) `6 8 10`

(D) `4 6 8 10`

Section II: Free-Response

The following are examples of the kinds of free-response questions found on the exam.

Question 1: Methods and Control Structures

This question involves the `MessageBuilder` class, which is used to generate a message based on a starting word. The `MessageBuilder` class contains a helper method, `getNextWord`. You will write a constructor and a method in the `MessageBuilder` class.

```
public class MessageBuilder
{
   private String message; // To be initialized in part (a)
   private int numWords;   // To be initialized in part (a)

   /**
    * Builds a message starting with the word specified by the
    * parameter and counts the number of words in the message,
    * as described in part (a)
    * Precondition: startingWord is a single word with no spaces.
    */
   public MessageBuilder(String startingWord)
   { /* to be implemented in part (a) */ }

   /**
    * Returns a word to follow the word specified by the
    * parameter or null if there are no remaining words.
    * Precondition: s is a single word with no spaces.
    * Postcondition: Returns an individual word with no spaces.
    */
   public String getNextWord(String s)
   { /* implementation not shown */ }

   /**
    * Returns an abbreviation for the instance variable message,
    * as described in part (b)
    * Preconditions: Each word in message is separated by a
    *                single space.
    *                message contains two or more words.
    * Postcondition: message is unchanged.
    */
   public String getAbbreviation()
   { /* to be implemented in part (b) */ }

   /* There may be instance variables, constructors, and methods
      that are not shown. */
}
```

(a) Write the `MessageBuilder` constructor, which uses a helper method to create a message starting with the word specified by the parameter `startingWord` and assigns the message to the instance variable `message`. The constructor also counts the number of words in the message and assigns the count to the instance variable `numWords`.

Each word in the message will be separated by a single space.

A helper method, `getNextWord`, has been provided to obtain words to be added to the message. Each call to `getNextWord` returns the next word in the message based on the word most recently added to the message. When there are no more words to be added to the message, `getNextWord` returns `null`. Consecutive calls to `getNextWord` are guaranteed to eventually return `null`.

Example 1

Consider the following calls to `getNextWord`, which are made within the `MessageBuilder` class. The return value of each call is used in the next call.

Method Call	Return Value
`getNextWord("the")`	`"book"`
`getNextWord("book")`	`"on"`
`getNextWord("on")`	`"the"`
`getNextWord("the")`	`"table"`
`getNextWord("table")`	`null`

Based on these return values, a call to the `MessageBuilder` constructor with argument `"the"` should set the instance variable message to `"the book on the table"` and should set the instance variable `numWords` to `5`.

Example 2

Consider the following calls to `getNextWord`, which are made within the `MessageBuilder` class. The return value of each call is used in the next call.

Method Call	Return Value
`getNextWord("good")`	`"morning"`
`getNextWord("morning")`	`"sunshine"`
`getNextWord("sunshine")`	`null`

Based on these return values, a call to the `MessageBuilder` constructor with argument `"good"` should set the instance variable `message` to `"good morning sunshine"` and should set the instance variable `numWords` to `3`.

Complete the `MessageBuilder` constructor. You must use `getNextWord` appropriately in order to receive full credit.

```
/**
 * Builds a message starting with the word specified by the
 * parameter and counts the number of words in the message,
 * as described in part (a)
 * Precondition: startingWord is a single word with no spaces.
 */
public MessageBuilder(String startingWord)
```

(b) Write the `getAbbreviation` method, which returns an abbreviation consisting of the first letter of each word in `message`. Assume that `message` consists of two or more words, each separated by a single space.

For example, if the value of `message` is `"as soon as possible"`, then `getAbbreviation()` should return `"asap"`.

Complete the `getAbbreviation` method.

```
/**
 * Returns an abbreviation for the instance variable message,
 * as described in part (b)
 * Preconditions: Each word in message is separated by a
 *                single space.
 *                message contains two or more words.
 * Postcondition: message is unchanged.
 */
public String getAbbreviation()
```

Question 2: Class Design

The `CupcakeMachine` class, which you will write, represents a cupcake vending machine, an automated machine that dispenses cupcakes. `CupcakeMachine` objects are created by calls to a constructor with two parameters.

- The first parameter is an `int` that represents the number of cupcakes that the vending machine has been stocked with. Assume that this value will be greater than or equal to `0`.
- The second parameter is a `double` that represents the cost, in dollars, per cupcake. Assume that this value will be greater than `0.0`.

The `CupcakeMachine` class contains a `takeOrder` method, which determines whether a cupcake order can be filled. A cupcake order can be filled if there are at least as many cupcakes in the vending machine as there are in the order.

A cupcake order is represented by a single `int` parameter to the `takeOrder` method. Assume that all values passed to the `takeOrder` method are positive.

If an order can be filled, the method updates the number of cupcakes remaining in the machine and returns a `String` containing information about the order. The returned string indicates the order number and the cost of the order, as shown in the following table. Order numbers begin at `1` and increase by `1` for each order filled.

If the order cannot be filled because the vending machine does not have enough cupcakes, the `takeOrder` method should return the message `"Order cannot be filled"`. In this case, the number of cupcakes available in the machine is unchanged and no order number is given to the order.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than `CupcakeMachine`.

Statement	Return Value (blank if no value)	Explanation
`String info;`		
`CupcakeMachine c1 = new CupcakeMachine(10, 1.75);`		`CupcakeMachine c1` is constructed with 10 cupcakes and a cost of $1.75 per cupcake.
`info = c1.takeOrder(2);`	`"Order number 1, cost $3.5"`	A customer orders 2 cupcakes for a total cost of 2 × $1.75. There are now 8 cupcakes remaining in the machine.
`info = c1.takeOrder(3);`	`"Order number 2, cost $5.25"`	A customer orders 3 cupcakes for a total cost of 3 × $1.75. There are now 5 cupcakes remaining in the machine.
`info = c1.takeOrder(10);`	`"Order cannot be filled"`	A customer attempts to order 10 cupcakes. The machine only has 5 cupcakes available, so no order is made.
`info = c1.takeOrder(1);`	`"Order number 3, cost $1.75"`	A customer orders 1 cupcake for a total cost of $1.75. There are now 4 cupcakes remaining in the machine.
`CupcakeMachine c2 = new CupcakeMachine(10, 1.5);`		`CupcakeMachine c2` is constructed with 10 cupcakes and a cost of $1.50 per cupcake.
`info = c2.takeOrder(10);`	`"Order number 1, cost $15.0"`	A customer orders 10 cupcakes for a total cost of 10 × $1.50. There are now 0 cupcakes remaining in the machine.

Write the complete `CupcakeMachine` class. Your implementation must meet all specifications and conform to the examples shown in the table.

Question 3: Data Analysis with `ArrayList`

The `ItemInfo` class is used to store information about an item at a store. A partial declaration of the `ItemInfo` class is shown.

```
public class ItemInfo
{
   /**
    * Returns the name of the item
    */
   public String getName()
   { /* implementation not shown */ }

   /**
    * Returns a value greater than 0.0 that represents the
    * cost of a single unit of the item, in dollars
    */
   public double getCost()
   { /* implementation not shown */ }

   /**
    * Returns true if the item is currently available and
    * returns false otherwise
    */
   public boolean isAvailable()
   { /* implementation not shown */ }

   /* There may be instance variables, constructors, and
      methods that are not shown. */
}
```

The `ItemInventory` class maintains an `ArrayList` named `inventory` that contains all items at the store. A partial declaration of the `ItemInventory` class is shown.

```
public class ItemInventory
{
   /** The list of all items at the store */
   private ArrayList<ItemInfo> inventory;

   /**
    * Returns the average cost of the available items
    * whose cost is between lower and upper, inclusive
    * Precondition: lower <= upper
    *               At least one available element of
    *               inventory has a cost between
    *               lower and upper, inclusive.
    *               No elements of inventory are null.
    */
   public double averageWithinRange(double lower, double upper)
   { /* to be implemented */}

   /* There may be instance variables, constructors, and methods
      that are not shown. */
}
```

Write the `ItemInventory` method `averageWithinRange`. The method should return the average cost of the available items in `inventory` whose cost is between the parameters `lower` and `upper`, inclusive.

Suppose `inventory` contains the following seven `ItemInfo` objects.

Name	"action figure"	"hair brush"	"frying pan"	"dish sponge"	"coffee mug"	"scarf"	"watch"
Cost	20.0	7.99	45.0	2.0	10.0	59.0	45.0
Is Available	true	true	true	false	true	true	false

For the inventory shown, `averageWithinRange(10.0, 50.0)` should return `25.0`, which is equal to the average cost of the available items within the specified range (a $20 action figure, a $45 frying pan, and a $10 coffee mug). Although the watch is within the specified range, it is not available.

Complete method `averageWithinRange`.

```
/**
 * Returns the average cost of the available items
 * whose cost is between lower and upper, inclusive
 * Precondition: lower <= upper
 *               At least one available element of
 *               inventory has a cost between
 *               lower and upper, inclusive.
 *               No elements of inventory are null.
 */
public double averageWithinRange(double lower, double upper)
```

Question 4: 2D Array

The `Appointment` class is used to store information about a person's scheduled appointments. A partial declaration of the `Appointment` class is shown.

```
public class Appointment
{
   /**
    * Returns the status of the appointment ("free", "busy", etc.)
    */
   public String getStatus()
   { /* implementation not shown */ }

   /**
    * Returns the room number of the appointment location
    */
   public int getRoomNumber()
   { /* implementation not shown */ }

   /* There may be instance variables, constructors, and methods
      that are not shown. */
}
```

The `Schedule` class maintains a two-dimensional array of appointments that represents a person's schedule. A partial declaration of the `Schedule` class is shown.

```
public class Schedule
{
   private Appointment[][] sched;

   /**
    * Returns the index of a column containing the fewest
    * occurrences of the status indicated by the parameter
    * target
    * Preconditions: sched is not null and no elements
    *                of sched are null.
    *                sched has at least one row and at
    *                least one column.
    */
   public int columnWithFewest(String target)
   { /* to be implemented */}

   /* There may be instance variables, constructors, and methods
      that are not shown. */
}
```

When an element of the two-dimensional array `sched` is accessed, the first index is used to specify the row and the second index is used to specify the column.

Write the `Schedule` method `columnWithFewest`. The method should return the index of a column in `sched` that contains the fewest occurrences of the parameter `target`. If there are multiple columns that have the fewest number of occurrences of `target`, any of their column indices can be returned.

Suppose `sched` has the following contents. For each element, the first value is the appointment status and the second value is the room number.

	0	1	2	3	4
0	"free" 100	"free" 100	"free" 100	"busy" 206	"busy" 204
1	"free" 100	"free" 100	"busy" 304	"busy" 206	"busy" 202
2	"hold" 201	"busy" 105	"busy" 205	"free" 100	"busy" 205
3	"busy" 204	"free" 100	"busy" 310	"hold" 110	"free" 100
4	"busy" 204	"hold" 201	"hold" 310	"busy" 105	"free" 100
5	"busy" 105	"busy" 208	"hold" 310	"busy" 105	"free" 100

For these contents of `sched`, the expected behavior of `columnWithFewest` is as follows.

- The call `columnWithFewest("busy")` should return `1` because `"busy"` appears two times in column `1` and more than two times in each of the other columns.
- The call `columnWithFewest("free")` should return either `2` or `3` because `("free")` appears one time in column `2`, one time in column `3`, and more than one time in each of the other columns.
- The call `columnWithFewest("hold")` should return `4` because `"hold"` appears zero times in column `4` and one or more times in each of the other columns.

Complete method `columnWithFewest`.

```
/**
 * Returns the index of a column containing the fewest
 * occurrences of the status indicated by the parameter
 * target
 * Preconditions: sched is not null and no elements
 *                of sched are null.
 *                sched has at least one row and at
 *                least one column.
 */
public int columnWithFewest(String target)
```

Answer Key and Question Alignment to Course Framework

Multiple-Choice Question	Answer	Skill	Learning Objective	Essential Knowledge
1	B	3.A	1.5.A	1.5.A.1 1.5.A.2
2	B	3.A	1.11.A	1.11.A.2 1.11.A.3
3	B	2.B	1.13.C	1.13.C.1 1.13.C.2
4	A	3.C	1.15.B	1.15.B.2 1.15.B.3
5	A	2.A	2.3.A	2.3.A.3 2.3.A.4
6	B	2.A	2.6.A	2.6.A.1 2.6.A.2
7	A	2.A	2.11.A	2.11.A.1
8	B	3.A	2.12.A	2.12.A.1
9	C	1.A	3.1.A	3.1.A.2 3.1.A.7
10	C	5.A	3.2.A	3.2.A.3
11	C	2.C	3.3.A	3.3.A.1 3.3.A.6
12	C	2.C	3.6.A	3.6.A.3
13	D	1.B	4.1.C	4.1.C.1
14	A	3.D	4.5.A	4.5.A.1
15	C	2.C	4.6.A	4.6.A.6 4.6.A.8
16	C	3.B	4.9.A	4.9.A.1
17	D	4.B	4.13.A	4.13.A.1
18	C	4.A	4.13.A	4.13.A.1
19	B	3.B	4.15.A	4.15.A.1 4.15.A.2
20	D	3.C	4.16.A	4.16.A.1 4.16.A.2

Free-Response Question	Skills	Learning Objectives
1	2.A, 2.B, 2.C	1.3.C, 1.14.A, 1.15.B, 2.7.B, 2.10.A
2	2.A, 2.B, 2.C	2.9.A , 3.3.A, 3.4.A, 3.5.A, 3.6.A
3	2.A, 2.B, 2.C	1.3.C, 1.14.A, 2.3.A, 4.9.A, 4.10.A
4	2.A, 2.B, 2.C	1.14.A, 2.2.A, 2.3.A, 4.12.A, 4.13.A

THIS PAGE IS INTENTIONALLY LEFT BLANK.

Applying the Scoring Criteria

Algorithm Points

Each question's scoring criteria includes one or two algorithm points, labeled "(*algorithm*)". Algorithm points involve applying all necessary steps of a solution in an appropriate order. Refer to the Scoring Guidelines for details on how to score algorithm points. Algorithm points can often be earned if steps assessed in other scoring criteria are present but incorrect. However, there are some situations in which algorithm points are not earned.

Algorithm points are NOT earned when:

- Required pieces are omitted
- The steps are assembled incorrectly
- Additional code not assessed in other scoring criteria makes the solution incorrect (e.g., printing to output, incorrect precondition check)
- A response confuses array/collection access (`[]` `.get`) when not otherwise assessed in the question
- Persistent data is modified (e.g., changing value referenced by parameter)
- A value is returned from a void method or constructor
- A provided method header is rewritten with a different number or type of parameters

Non-Algorithm Points

All points not labeled "(*algorithm*)" are assessed with a narrower focus and can be earned even if other errors are present in the response.

Errors to Ignore

For both algorithm and non-algorithm points, some errors are considered minor. The point may still be earned even when the errors are present. Such errors include:

- Extraneous code with no side effect (e.g., valid precondition check, no-op)
- Spelling/case discrepancies where there is no ambiguity*
- Local variable not declared (unless scoring guidelines specifically require it)
- Local variable declared inside a block but used outside the block
- `private` or `public` qualifier on a local variable
- Missing `public` qualifier on a class or constructor header
- Provided method header rewritten with different variable names
- Keyword used as an identifier
- Common mathematical symbols used for operators (× • ÷ ≤ ≥ <> ≠)
- `[]` vs. `()` vs. `<>` confusion
- Using `=` instead of `==` and vice versa
- `length/size` confusion for array, `String`, or `ArrayList`; with or without `( )`
- Extraneous `[]` when referencing entire array
- Extraneous size in array declaration (e.g., `int[size] nums = new int[size];`)
- Using `[i,j]` instead of `[i][j]`
- Missing `;` where structure clearly conveys intent
- Missing `{ }` where indentation clearly conveys intent
- Missing `( )` on parameter-less method or constructor invocations
- Missing `( )` around `if`, `for`, or `while` conditions

Spelling and case discrepancies for identifiers fall under the "Errors to Ignore" category only if the correction can be **unambiguously inferred from context as in the example "ArayList" instead of "ArrayList". As a counterexample, if the code declares "`int G=99, g=0;`", then uses "`while (G < 10)`" instead of "`while (g < 10)`", the context does **not** allow the reader to assume the use of the lowercase variable.*

Question 1: Methods and Control Structures — 7 points

Canonical solution

(a) **4 points**

```
public MessageBuilder(String startingWord)
{
   message = startingWord;
   String w = getNextWord(startingWord);
   numWords = 1;

   while(w != null)

   {
      message += " " + w;
      numWords++;
      w = getNextWord(w);
   }
}
```

3 points

(b)

```
public String getAbbreviation()
{
   String temp = message;
   String result = temp.substring(0, 1);

   while (temp.indexOf(" ") >= 0)
   {
      int i = temp.indexOf(" ");
      temp = temp.substring(i + 1);
      result += temp.substring(0, 1);
   }
   return result;
}
```

(a) `MessageBuilder`

	Scoring Criteria	Decision Rules	
1	Calls `getNextWord` on the value of `startingWord`	Responses **will not** earn the point if they • make any incorrect call to `getNextWord`	**1 point**
2	Loops until no more words remaining	Responses **will not** earn the point if they • make an incorrect comparison to `null`	**1 point**
3	Increments a count of the number of words accessed (in the context of a loop)	Responses **can** still earn the point even if they • fail to assign the count to `numWords` • fail to initialize `numWords`	**1 point**
4	`message` is correctly built from `startingWord` and the strings obtained from `getNextWord` and `numWords` is assigned the correct count (*algorithm*)	Responses **will not** earn the point if they • fail to initialize `message` • calculate an incorrect count • include an additional space before the first word or after the last word	**1 point**
		Total for part (a)	**4 points**

(b) `getAbbreviation`

	Scoring Criteria	Decision Rules	
5	Loop accesses all necessary parts of `message`	Responses **can** still earn the point even if they • fail to extract the first letter of the word in a word-based loop	**1 point**
6	Identifies a letter that appears after a space within message (and all `String` method calls are syntactically valid)		**1 point**
7	Returns string consisting of the first letter of each word in `message` (*algorithm*)	Responses **will not** earn the point if they • modify `message` or `numWords`	**1 point**
		Total for part (b)	**3 points**
		Total for question 1	**7 points**

Note: Solutions to part (b) that iterate over the string character-by-character or use the `split` method are valid and can be scored using these scoring guidelines.

Question 2: Class Design — 7 points

Canonical solution

7 points

```
public class CupcakeMachine
{
   private int availCupcakes;
   private double cupcakeCost;
   private int orderNum;

   public CupcakeMachine(int num, double cost)
   {
      availCupcakes = num;
      cupcakeCost = cost;
      orderNum = 1;
   }

   public String takeOrder(int quantity)
   {
      String message = "Order cannot be filled";

      if (quantity <= availCupcakes)
      {
         availCupcakes -= quantity;
         double cost = quantity * cupcakeCost;
         message = "Order number " + orderNum
                   + ", cost $" + cost;
         orderNum++;
      }

      return message;
   }
}
```

`CupcakeMachine`

	Scoring Criteria	Decision Rules	
1	Declares class header: `class CupcakeMachine`	Responses **will not** earn the point if they • declare the class as `private` • include extraneous code outside the class	**1 point**
2	Declares `private` instance variables of appropriate types to store the number of cupcakes, the order number, and the cost per cupcake	Responses **will not** earn the point if they • declare any instance variable `static` • declare a variable outside the class	**1 point**

3	Declares constructor header:
	`CupcakeMachine(int ___,`
	`String ___)`
		Responses **will not** earn the point if they
		• declare the constructor as `private`
			1 point

4	Constructor assigns correct initial values to all instance variables	Responses **can** still earn the point even if they • initially assign an incorrect order number, as long as the algorithm corrects the order number before the first string is built Responses **will not** earn the point if they • fail to declare instance variables for number of cupcakes, order number, and cost per cupcake	**1 point**
5	Declares method header: `String takeOrder(int ___)`	Responses **will not** earn the point if they • use incorrect method name • declare method as something other than `public`	**1 point**
6	Builds message based on `int` parameter, updates number of available cupcakes, and updates the order number appropriately (*algorithm*)	Responses **can** still earn the point even if they • fail to return the message • make minor spelling errors, as long as both `String` literal parts are included • declare instance variables outside the class, or in the class within a method or constructor Responses **will not** earn the point if they • fail to handle the case when an order cannot be filled • build correct messages for cases when an order can and cannot be filled, but return something else	**1 point**
7	Returns appropriate constructed string message	Responses **can** still earn the point even if they • return an incorrect message Responses **will not** earn the point if they • return a single `String` literal with no computation • return a non-`String` value • fail to return a value in some cases	**1 point**
		Total for question 2	**7 points**

Question 3: Data Analysis with `ArrayList` — 5 points

Canonical solution

5 points

```
public double averageWithinRange(double lower, double upper)
{
   double sum = 0.0;
   int count = 0;

   for (ItemInfo it : inventory)
   {
      if (it.isAvailable()
          && it.getCost() >= lower && it.getCost() <= upper)
      {
         sum += it.getCost();
         count++;
      }
   }
   return sum / count;
}
```

`averageWithinRange`

	Scoring Criteria	Decision Rules	
1	Accesses all items in `inventory` (*no bounds errors*)	Responses **can** still earn the point even if they • fail to use the accessed value Responses **will not** earn the point if they • use an indexed loop with correct bounds but fail to access an item inside the loop • access the elements of `inventory` incorrectly • use array syntax to access `ArrayList` elements (`[ ]`)	**1 point**
2	Calls `getCost` and `isAvailable` on an element from the list	Responses **can** still earn the point even if they • use array syntax to access `ArrayList` elements (`[ ]`) Responses **will not** earn the point if they • call either method on an object other than `ItemInfo`	**1 point**
3	Checks whether the cost of an item is between `lower` and `upper`, inclusive	Responses **can** still earn the point even if they • access `ItemInfo` or `ArrayList` incorrectly • fail to check whether the item is available • call `getCost` incorrectly Responses **will not** earn the point if they • use incorrect syntax or logic for the condition (e.g., `and` in place of `&&`, `<` in place of `<=`) • fail to call `getCost`	**1 point**
4	Declares a `double` sum variable and accumulates sum of costs from `inventory` within a loop	Responses **can** still earn the point even if they • compute a sum of incorrectly identified costs • call `getCost` incorrectly • fail to initialize the sum variable Responses **will not** earn the point if they • fail to call `getCost`	**1 point**
5	Returns correct average cost of available items within range (*algorithm*)	Responses **can** still earn the point even if they • access `ItemInfo` or `ArrayList` incorrectly • use array syntax to access `ArrayList` elements (`[ ]`) • calculate an average based on an incorrect sum, as long as the sum and count are consistent Responses **will not** earn the point if they • fail to initialize either the sum or the count variable • include unavailable items in the sum • fail to check range or combine range and availability checks incorrectly • compute the average by dividing anything other than the computed count • fail to return the computed average	**1 point**
		Total for question 3	**5 points**

Question 4: 2D Array — 6 points

Canonical solution

6 points

```
public int columnWithFewest(String target)
{
   int minCol = 0;
   int minCount = sched.length;
   for (int col = 0; col < sched[0].length; col++)
   {
      int count = 0;
      for (int row = 0; row < sched.length; row++)
      {
         if (sched[row][col].getStatus().equals(target))
         {
            count++;
         }
      }
      if (count < minCount)
      {
         minCol = col;
         minCount = count;
      }
   }
   return minCol;
}
```

`columnWithFewest`

	Scoring Criteria	Decision Rules	
1	Accesses all elements of `sched` (*no bounds errors*)	Responses **will not** earn the point if they • fail to access elements of `sched` correctly	**1 point**
2	Calls `getStatus` to obtain a status of an `Appointment` object	Responses **can** still earn the point even if they • access elements of `sched` out of bounds • fail to access elements of `sched` correctly Responses **will not** earn the point if they • call `getStatus` incorrectly	**1 point**
3	Compares a status with `target`	Responses **can** still earn the point even if they • access a status incorrectly Responses **will not** earn the point if they • use `==` to compare statuses	**1 point**
4	Compares identified counts of target value for two columns	Responses **can** still earn the point even if they • determine the number of elements in each row instead of each column • compute an incorrect count • reverse an inequality when comparing counts Responses **will not** earn the point if they • fail to maintain the count of a previous column to compare to the current column	**1 point**
5	Determines the number of elements in each accessed column that match target value (*algorithm*)	Responses **can** still earn the point even if they • determine the number of elements in each row instead of each column Responses **will not** earn the point if they • fail to reset the count when advancing to the next column • fail to initialize a count	**1 point**
6	Returns index of column with smallest computed count (*algorithm*)	Responses **can** still earn the point even if they • compute the count for individual columns incorrectly Responses **will not** earn the point if they • confuse rows with columns • fail to initialize the minimum count • fail to initialize the index • fail to return the computed value • return the count instead of the index • reverse an inequality when comparing counts	**1 point**
		Total for question 4	**6 points**

THIS PAGE IS INTENTIONALLY LEFT BLANK.

AP COMPUTER SCIENCE A

Appendix

elloWorld.ja
*/
public class He
{
pu
ain

Java Quick Reference

This table contains accessible methods from the Java library that may be included on the AP Computer Science A Exam.

Class Constructors and Methods	Explanation
String Class	
`String(String str)`	Constructs a new `String` object that represents the same sequence of characters as `str`
`int length()`	Returns the number of characters in a `String` object
`String substring(int from, int to)`	Returns the substring beginning at index `from` and ending at index `to − 1`
`String substring(int from)`	Returns `substring(from, length())`
`int indexOf(String str)`	Returns the index of the first occurrence of `str`; returns `-1` if not found
`boolean equals(Object other)`	Returns `true` if `this` corresponds to the same sequence of characters as `other`; returns `false` otherwise
`int compareTo(String other)`	Returns a value < 0 if `this` is less than `other`; returns zero if `this` is equal to `other`; returns a value > 0 if `this` is greater than `other`. Strings are ordered based upon the alphabet.
`String[] split(String del)`	Returns a `String` array where each element is a substring of `this String`, which has been split around matches of the given expression `del`
Integer Class	
`Integer.MIN_VALUE`	The minimum value represented by an `int` or `Integer`
`Integer.MAX_VALUE`	The maximum value represented by an `int` or `Integer`
`static int parseInt(String s)`	Returns the `String` argument as an `int`
Double Class	
`static double parseDouble(String s)`	Returns the `String` argument as a `double`
Math Class	
`static int abs(int x)`	Returns the absolute value of an `int` value
`static double abs(double x)`	Returns the absolute value of a `double` value
`static double pow(double base, double exponent)`	Returns the value of the first parameter raised to the power of the second parameter
`static double sqrt(double x)`	Returns the nonnegative square root of a `double` value
`static double random()`	Returns a `double` value greater than or equal to `0.0` and less than `1.0`
ArrayList Class	
`int size()`	Returns the number of elements in the list
`boolean add(E obj)`	Appends `obj` to end of list; returns `true`
`void add(int index, E obj)`	Inserts `obj` at position `index (0 <= index <= size)`, moving elements at position `index` and higher to the right (adds 1 to their indices) and adds 1 to size
`E get(int index)`	Returns the element at position `index` in the list
`E set(int index, E obj)`	Replaces the element at position `index` with `obj`; returns the element formerly at position `index`
`E remove(int index)`	Removes element from position `index`, moving elements at position `index + 1` and higher to the left (subtracts 1 from their indices) and subtracts 1 from size; returns the element formerly at position `index`

File Class	
`File(String pathname)`	The `File` constructor that accepts a `String` pathname

Scanner Class	
`Scanner(File f)`	The `Scanner` constructor that accepts a `File` for reading
`int nextInt()`	Returns the next `int` read from the file or input source if available. If the next `int` does not exist or is out of range, it will result in an `InputMismatchException`.
`double nextDouble()`	Returns the next `double` read from the file or input source. If the next `double` does not exist, it will result in an `InputMismatchException`.
`boolean nextBoolean()`	Returns the next `boolean` read from the file or input source. If the next `boolean` does not exist, it will result in an `InputMismatchException`.
`String nextLine()`	Returns the next line of text as a `String` read from the file or input source; can return the empty string if called immediately after another `Scanner` method that is reading from the file or input source
`String next()`	Returns the next `String` read from the file or input source
`boolean hasNext()`	Returns `true` if there is a next item to read in the file or input source; `false` otherwise
`void close()`	Closes this scanner

Object Class
`boolean equals(Object other)`
`String toString()`

collegeboard.org

 APD-531